

COMSATS UNIVERSITY ISLAMABAD, ATTOCK CAMPUS

Department of Computer Sciences



Course Name: Information Security Lab
Project Report

Submitted by: Muhammad Usama (FA24-BSE-062)
Muhammad Sufiyan (FA24-BSE-049)

Instructor: Mam Ambreen Gul

Date: 18 May, 2026

File Integrity Checker

Abstract:

A File Integrity Checker is a security mechanism used to verify whether files remain unchanged, authentic, and free from unauthorized modifications. It checks whether file content has been altered intentionally or accidentally. The system generates a digital fingerprint called a hash value and compares it with an original stored value. The purpose is to maintain data integrity and provide trust in digital information.

Introduction:

Data security is one of the most important concerns in computer systems. Files can be modified due to malware attacks, unauthorized users, accidental changes, hardware failure, transmission errors, or corruption. Users and organizations need methods to verify that files remain exactly as intended. File Integrity Checkers solve this problem by using cryptographic hashing techniques.

What is File Integrity?

File integrity means that data remains complete, accurate, and unchanged from its original state. Integrity ensures that no unauthorized modifications occur during storage, processing, or transmission. If even a single character changes inside a file, integrity is considered compromised.

Problem Statement:

Users often cannot determine whether a file has been altered after storage or transfer. Traditional methods such as checking file size or date modified are unreliable because attackers may manipulate them. Therefore, a stronger method is required to verify file authenticity.

Objectives of File Integrity Checker:

1. Detect unauthorized file modifications.
2. Identify accidental corruption.
3. Verify software authenticity.
4. Protect critical files.
5. Maintain trust in digital systems.
6. Provide security monitoring.

Theory of Hashing:

A hash function converts input data of any size into a fixed-length output known as a hash or digest. Hash functions possess several properties:

- Deterministic: same input produces same output.
- Fast computation.
- One-way function.
- Avalanche effect: tiny changes create major hash changes.
- Collision resistance.

SHA-256 Algorithm:

SHA-256 belongs to the SHA-2 family developed by NIST. It generates a 256-bit digest represented by a 64-character hexadecimal string. It is widely used because it provides strong security and resistance against collision attacks.

Working Mechanism

- Step 1: User selects a file.
- Step 2: System reads file data.
- Step 3: SHA-256 calculates a hash.
- Step 4: Original hash value is saved.
- Step 5: During later verification, the file hash is recalculated.
- Step 6: New and old values are compared.
- Step 7: If equal → file unchanged.
- Step 8: If different → file modified.

Algorithm

Start
Input file path
Read file in binary mode
Generate SHA-256 hash
Store hash
Recalculate hash during verification
Compare values
Display result
End

Python Implementation Concept:

The project commonly uses hashlib library in Python. Files are read in chunks instead of loading complete data into memory. Chunk reading improves efficiency for large files.

Example Scenario:

Suppose a file named report.txt contains original content. The generated SHA-256 hash is stored. Later, if someone changes one word or adds a single character, the new hash becomes completely different due to the avalanche effect. The File Integrity Checker immediately identifies modification.

Advantages:

- High security
- Fast verification process
- Detects hidden changes
- Easy implementation
- Useful in cybersecurity
- Supports automation
-

Limitations:

- Does not recover files.
- Requires secure storage of original hashes.
- Cannot identify who modified the file.
- Large systems may require database storage.

Applications:

1. Operating system security.
2. Antivirus systems.
3. Digital forensics.
4. Software verification.
5. Cloud storage systems.
6. Banking systems.
7. Military and enterprise security.

Future Enhancements:

Future versions can include GUI interfaces, databases, email alerts, real-time monitoring, cloud integration, and support for multiple hashing algorithms.

Code:

```
# Fake File Integrity Checker
# IS Lab Project
# Checks whether a file has been changed using SHA-256 hash

import hashlib
import os

# Function to calculate SHA-256 hash of a file
def calculate_hash(file_path):
    sha256 = hashlib.sha256()

    try:
        with open(file_path, "rb") as file:
```

```

        while True:
            data = file.read(4096)

            if not data:
                break

            sha256.update(data)

        return sha256.hexdigest()

    except FileNotFoundError:
        return None

# Function to save original hash
def save_hash(file_path, hash_value):
    hash_file = file_path + ".hash"

    with open(hash_file, "w") as file:
        file.write(hash_value)

    print("Original hash saved successfully.")

# Function to read saved hash
def load_hash(file_path):
    hash_file = file_path + ".hash"

    try:
        with open(hash_file, "r") as file:
            return file.read()

    except FileNotFoundError:
        return None

# Function to verify file integrity
def verify_integrity(file_path):

    current_hash = calculate_hash(file_path)

    if current_hash is None:
        print("File not found!")
        return

    saved_hash = load_hash(file_path)

```

```

    if saved_hash is None:
        print("No previous hash found.")
        print("Creating baseline hash...")

        save_hash(file_path, current_hash)

    else:

        if current_hash == saved_hash:
            print("File Integrity Status: SAFE")
            print("No changes detected.")

        else:
            print("WARNING!")
            print("File has been modified.")
            print("Integrity check FAILED")

# Main Menu
while True:

    print("\n===== Fake File Integrity Checker =====")
    print("1. Check File")
    print("2. Exit")

    choice = input("Enter choice: ")

    if choice == "1":

        path = input("Enter file path: ")

        verify_integrity(path)

    elif choice == "2":
        print("Program Closed")
        break

    else:
        print("Invalid choice")

```

Output:

Case 1: First Time File Check - No Hash Exists

```
===== Fake File Integrity Checker =====  
1. Check File  
2. Exit  
Enter choice: 1  
Enter file path: test.txt  
  
No previous hash found.  
Creating baseline hash...  
Original hash saved successfully.
```

Case 2: File Not Modified - SAFE State

```
===== Fake File Integrity Checker =====  
1. Check File  
2. Exit  
Enter choice: 1  
Enter file path: test.txt  
  
File Integrity Status: SAFE  
No changes detected.
```

Case 3: File Modified - Tampering Detected

```
===== Fake File Integrity Checker =====  
1. Check File  
2. Exit  
Enter choice: 1  
Enter file path: test.txt  
  
WARNING!  
File has been modified.  
Integrity check FAILED
```


Case 4: File Does Not Exist

```
===== Fake File Integrity Checker =====  
1. Check File  
2. Exit  
Enter choice: 1  
Enter file path: wrongfile.txt  
  
File not found!
```

Case 5: Exit Program

```
===== Fake File Integrity Checker =====  
1. Check File  
2. Exit  
Enter choice: 2  
  
Program Closed
```